

개인 Activity - 임채현_개발해석 보고서

6조 임채현 | charGPT(문자 단위 미니 GPT) Baseline 개선 실습

1. Transformer는 어떻게 다음 글자를 예측할까

토큰화 — 글자 하나가 곧 단위다. charGPT는 단어가 아니라 글자 하나를 최소 단위로 쓴다. 김유정 작품 97,709자에 나온 글자 1,272개가 모델의 글자 목록이 되고, 각 글자에 번호가 붙는다. 만들기는 간단하지만 한글은 글자 종류가 많아 목록이 커지고 같은 문장에도 단위가 훨씬 많이 든다. 학습 초반 생성문이 "할냥엿롱굶대전투..."처럼 나오는 것도, 단어가 아니라 글자를 하나씩 붙여 만들기 때문이다.

임베딩 — 번호에 의미를 입힌다. 번호는 순번일 뿐 뜻이 없어서, 각 번호를 `N_EMBD` 길이의 숫자 묶음으로 바꾸고 학습하면서 다듬는다. 여기에 "몇 번째 자리인가"도 더한다. Attention은 입력을 순서 없는 덩어리로 다루기 때문에, 자리 정보를 넣지 않으면 "가나다"와 "다나가"를 똑같이 본다.

Self-Attention — 앞 글자 중 뭘 볼지 고른다. 글자마다 Query, Key, Value를 만들고, Query와 Key를 맞춰보며 "다음을 맞히려면 앞의 어떤 글자를 얼마나 참고할까"를 점수로 매겨 그 비율대로 Value를 섞는다(점수가 커지지 않게 `1/√head_dim` 으로 줄인다). 이걸 `N_HEAD` 개로 나눠 돌리는 건 조사 호응, 따옴표 대화처럼 서로 다른 관계를 나눠 보게 하려는 것이고, 그래서 `N_EMBD` 는 `N_HEAD` 로 나누어떨어져야 한다.

Causal Mask — 뒤를 못 보게 가린다. 푸는 문제가 "다음 글자 맞추기"인데 뒤쪽까지 보면 답을 미리 보는 셈이다. 그래서 자기 자리보다 뒤에 있는 글자의 점수를 $-\infty$ 로 만들어 참고 비율을 0으로 없앤다. 성능을 올리는 장치가 아니라 **연습과 실전을 같은 조건으로 맞추는 장치**이며, 꺾을 때 무슨 일이 나는지는 2.2에 적었다.

Logits와 Loss — 점수를 매기고 틀린 만큼 고친다. 마지막에 나온 값을 글자 1,272개에 대한 점수로 펼치고, 정답 글자에 얼마나 높은 점수를 줬는지 재는 게 loss다. `exp(loss)` 인 perplexity는 "후보를 평균 몇 개까지 좁혔나"로 읽는다. loss가 나오면 거꾸로 따라가며 어떤 값이 오답에 기여했는지 구하고(역전파) AdamW가 그만큼씩 고치는데, 이 한 바퀴를 `MAX_ITERS` 번 반복하는 게 학습이다.

생성 — 한 글자씩 이어 붙인다. 점수를 temperature로 나눠 뽀족하게/뽀뽀하게 만들고 top-k로 후보를 몇 개만 남긴 뒤 하나를 뽑는다. 그리고 **뽑은 글자를 입력 뒤에 붙여 다시 넣고 또 뽑는다.** 이 반복이 우리가 본 생성문이고, 입력이 `BLOCK_SIZE` 보다 길어지면 뒤쪽만 남기고 자른다.

2. 실험에서 관찰한 것

2.1 조별 실험 — 6조가 바꾼 건 2개

이 절 수치는 6조 조별 보고서 5·6장의 결과다.

실험	조건	결과
모델 크기 (MAX_ITERS=100 고정)	Small 3/3/96 · Base 6/6/192 · Large 12/12/384	Loss 4.009 → 3.685 → 3.657 학습시간 3.8s → 17.4s → 52.2s
학습 반복 수 (Base 구조 고정)	100 · 500 · 2000	Train 4.04 → 3.52 → 3.04 Val 4.25 → 3.79 → 3.62

- **키운 만큼 돌아오지 않았다.** Base에서 Large로 갈 때 Loss는 0.028 줄었는데 학습 시간은 3배가 됐다.
- **반복을 늘리자 Train과 Val이 벌어졌다.** 차이가 100회 0.21에서 2000회 0.58이 됐다. 줄어든 loss만큼 실력이 는 게 아니라 외우는 쪽으로 가고 있다는 신호다.
- **생성문은 loss보다 천천히 좋아졌다.** 100회에서는 "그 / 이 / 하다"처럼 짧은 말만 반복했고, 500회부터 따옴표 대화 모양이 잡혔다. 2000회에서는 "아침을 제 돈 이 제 제 어 찌면 나?" 같은 김유정 특유의 말투가 조금씩 보였다.

2.2 개인 후속 실험 — 6개 변수 전부

발표 뒤, 학습에 쓰지 않고 남겨둔 검증용 데이터(뒤 10%)를 모든 실험에 똑같이 붙이고 6개를 다 돌렸다. 해석이 달라진 곳이 세 군데다.

- **모델 크기를 셋으로 나눠 보니 방향이 없었다.** `N_LAYER` =12(Val 3.5049), `N_HEAD` =3(3.5178), `N_EMBD` =192(3.5392)가 각각 제일 좋았고, 키울수록 좋아지거나 나빠지는 흐름이 아니었다. 특히 `N_EMBD` =384는 Train이 2.0830까지 떨어졌는데 Val은 3.6335로 나빠졌다. 커진 만큼을 실력이 아니라 외우는 데 써버린 것이다.
- **2000회 안에 이미 꺾이는 지점이 있었다.** 조별 실험은 마지막 값만 비교해 Val이 계속 좋아지는 것처럼 보였다. 학습 도중의 Val을 촘촘히 찍어보니 **450회에서 3.5072로 제일 낮았다**가 마지막엔 5.1613까지 되올라갔다. 코드가 매 스텝 데이터를 무작위로 다시 뽑아 쓰는 방식이라 2000회면 학습에 쓴 글을 **약 186번 읽은 셈**이다. dropout 0.1을 켜둔 상태에서도 외워버릴 양이다.
- **Causal Mask를 꺾더니 Val이 0.0841(perplexity 1.09)까지 떨어졌다.** 숫자만 보면 압도적인데 생성문은 "...늘지늘지지지지지지..."로 완전히 망가졌다. 가림막이 없으면

연습할 땐 뒷글자를 볼 수 있지만 실제로 쓸 땐 뒷글자가 아직 없다. **loss가 낮다고 좋은 모델이 아니라는 것**을 가장 확실하게 보여준 경우다.

이 밖에 **BLOCK_SIZE** 는 128보다 64(3.4790)가 좋았다. **DATA_FRACTION** =0.25에서는 Train 1.1622 / Val 5.4106으로 두 값이 정반대로 움직였는데, 데이터가 1/4이면 같은 횟수를 돌려도 같은 구간을 4배 자주 보게 되어 더 빨리 외워지기 때문이다.

2.3 종합 — 발목을 잡은 건 구조가 아니라 데이터였다

Val이 움직인 폭만 모아보면 답이 분명하다. 구조를 바꾼 넷

(**N_LAYER** · **N_HEAD** · **N_EMBD** · **BLOCK_SIZE**)은 3.4790~3.6335로 **0.15 안에서만** 움직였는데, 데이터 양 하나가 3.5392~5.4106으로 **1.87을 움직였다**. 약 12배 차이다. 제일 좋은 값만 골라 조합해 다시 학습했을 때도 3.5392 → 3.4690, 0.07밖에 못 줄였다.

여섯 개를 다 최적화해 얻은 게 데이터를 늘렸을 때 효과의 1/27 수준이니, 이 정도 데이터에서 구조를 다듬어 올릴 수 있는 한계는 이미 가깝다. 게다가 88K자까지 늘렸는데도 Val은 **여전히 가파르게 좋아지는 중**이었다. 곡선이 늪지 않았다는 건 아직 데이터가 모자란 구간이고, 더 넣으면 더 좋아질 여지가 크다는 뜻이다.

N_EMBD =384에서 Train만 떨어진 것도, 450회 넘어 Val이 되올라간 것도 결국 같은 얘기다. 모델을 키우거나 오래 돌려 생긴 여유분이 실력이 아니라 **짧은 글을 더 빨리 외우는 데** 쓰인 것이다.

3. 결과에서 확인한 한계

- **재는 방식이 거칠다**. 코드가 매 스텝 64개 샘플의 loss를 그대로 쌓고 "최종 Loss"는 그중 마지막 값이다. 게다가 같은 seed(3407)로 한 번씩만 돌렸다. 3.5049와 3.5392 같은 0.03 차이가 진짜인지 우연인지 확인할 방법이 없다.
- **조별 실험은 한 번에 여러 개를 바꿨고, 재본 지점도 띄엄띄엄했다**. 세 값을 같이 올려 깊이·폭·헤드 수 중 뭐가 효과인지 가릴 수 없었고, Val도 세 지점의 마지막 값만 봤다. 개인 실험에서 하나씩 따로 돌리고 도중 값을 찍어 이 문제는 해소했다. 다만 **N_LAYER** 만은 코드에서 초기값이 $std = 0.02/\sqrt{2 \cdot n_layer}$ 로 층수에 묶여 있어, 따로 돌려도 초기값 크기가 같이 바뀌는 건 남는다.
- **DATA_FRACTION** 은 학습에서 못 본 글자를 만든다. 글자 목록 1,272개는 전체 corpus 기준으로 고정인데 글은 앞에서부터 잘라 쓰므로, 0.25 조건에서는 학습 중 한 번도 나오지 않는 글자가 생긴다. 그 글자들이 검증용 데이터에 등장하면 loss를 더 밀어올린다.
- **글이 너무 짧다**. 97,709자에 글자 종류가 1,272개라 한 글자당 평균 77번밖에 안 나온다. 배울 거리가 구조적으로 부족하고, 2.3에서 본 대로 이게 모든 결과를 눌러버린 가장 큰 제약이다.

4. 실제 LLM과의 차이, 그리고 개선 방향

	이번 미니 모델	실제 LLM
단위	글자 1,272개	자주 쓰는 덩어리 수만 개
데이터	97,709자 (김유정 8편)	비교가 안 되게 많은 양
크기	최대 12층 / 384차원	훨씬 깊고 넓음
학습	학습률 고정, 최대 2000회	학습률 조절, 훨씬 긴 학습
이후	없음	지시 따르기 학습 등 추가 단계

구조는 같은 뼈대인데 규모와 뒷단계가 없다. 그래서 이 모델은 "그럴듯한 글자 이어붙이기"까지만 하고 뜻을 이해하거나 지시를 따르지는 못한다. 2.3에서 데이터 양만 유일하게 크게 움직였던 것도 이 차이의 축소판이다.

이번 실습으로 가장 크게 이해한 건 **loss가 모델 실력의 대리 지표일 뿐이라는 점**이다. Causal Mask를 꺾을 때도, 데이터를 줄였을 때도 loss는 좋아졌지만 실제 성능은 반대였다.

(1) Val을 계속 보면서 알아서 멈추게 한다. 1조 밤티는 Val Loss와 Train과의 차이를 같이 추적해 **600회쯤**을 과적합 시작 지점으로 잡고 Early Stopping을 제안했다. 내 실험에서 제일 좋았던 건 450회다. 모델도 데이터 나누는 방식도 달라 숫자는 다르지만 **500회 안팎에서 꺾인다**는 결론이 따로 나왔으니 우연은 아니라고 본다. 반대로 6조가 쓴 500회는 근거가 있어 고른 게 아니라 운이 좋았던 값이다.

(2) 전체 크기를 맞춰놓고 모양만 비교한다. 1조는 파라미터 총 개수를 맞춘 뒤 "깊고 좁은 모델 vs 얇고 넓은 모델"을 비교했다. 세 값을 한꺼번에 키우면 "크게 하면 좋아진다"는 뻔한 결론뿐이지만, 총 개수를 고정하면 **같은 예산을 깊이에 쓸까 폭에 쓸까**라는 실제 고민에 답할 수 있다. 내가 하나씩만 바꾼 건 첫 단계였을 뿐 총량까지 맞추진 못했다.

(3) loss 하나만 보지 말고 생성문도 같이 본다. 2조 연봉2조가 말한 "낮은 loss ≠ 좋은 모델"은 내 Causal Mask 실험에서 그대로 확인됐다. Val 0.0841이라는 최고 기록이 완전히 망가진 생성문과 짝이었기 때문이다. 앞으로는 서로 다른 글자가 얼마나 나왔는지, 같은 두 글자 짝이 얼마나 반복됐는지 같은 수치를 생성문 원문과 함께 loss 옆에 놓아야 한다.

(4) 데이터를 늘리는 게 먼저, tokenizer 교체가 그다음. 2.3에서 본 대로 가장 크게 효과를 낸 건 데이터 양이고 88K자에서도 아직 좋아지는 중이었다. 작품을 더 늘리는 게 어떤 구조 튜닝보다 우선이다. 그다음이 tokenizer다. 글자 하나씩 쓰면 같은 내용에 자리가 많이 들어 **BLOCK_SIZE**를 늘려도 실제 참고 범위는 몇 어절뿐이다(128보다 64가 좋았던 것도 이 때문으로 보인다). BPE처럼 자주 나오는 덩어리를 묶으면 같은 데이터와 계산량으로도 더 나은 결과를 기대할 수 있다.